

RELATÓRIO TÉCNICO DE AUDITORIA

WHEngenharia — Sistema ERP



ALTO RISCO
Classificação Final do Sistema

Data da Auditoria:	Abril de 2026
Classificação:	Confidencial
Repositório:	MarqSolucoes/REPO-TEMPORARIO
Escopo:	Frontend (Vue.js 2) + Backend (ASP.NET Core) + Banco de Dados (SQL Server)

Análise independente e crítica do código-fonte. Gerado com base em inspeção direta de todos os arquivos disponíveis no repositório. Itens não verificáveis por ausência de ambiente de execução estão explicitamente sinalizados.

. SUMÁRIO EXECUTIVO

1.	Resumo Executivo (não técnico)
2.	Visão Geral da Arquitetura
3.	Análise do Frontend
4.	Análise do Backend
5.	Banco de Dados
6.	Segurança
7.	Performance
8.	DevOps e Processo
9.	Problemas Críticos (ALTA PRIORIDADE)
10	Melhorias Recomendadas
11	Classificação Final do Sistema

1. Resumo Executivo (não técnico)

■ Estado Geral do Sistema

O sistema está em **estado crítico de risco**. O ERP da WHEngenharia funciona operacionalmente, mas apresenta falhas graves de segurança que expõem dados confidenciais da empresa, credenciais de acesso ao banco de dados e senhas de e-mail diretamente no código-fonte — disponíveis para qualquer pessoa com acesso ao repositório. Isso representa uma violação de segurança imediata e ativa.

Principais Riscos para a Diretoria

■ Senhas e credenciais de produção no repositório

Qualquer funcionário ou ex-funcionário com acesso ao repositório pode conectar diretamente ao banco de dados de produção agora.

■ Empresa refém do fornecedor/desenvolvedor atual

A infraestrutura está hospedada em conta pessoal (osvaldotneto-t.eastus2.cloudapp.azure.com), sem documentação de deploy, sem pipeline automatizado e sem processo de transferência documentado. Uma saída do fornecedor compromete a continuidade do sistema.

■ Bug confirmado de cálculo financeiro no Fluxo de Caixa

Duplicação de valores em relatórios financeiros. Decisões tomadas com base nesses dados estão potencialmente erradas.

■ Zero testes automatizados

Qualquer mudança no sistema pode quebrar funcionalidades existentes sem que ninguém perceba antes dos usuários.

2. Visão Geral da Arquitetura

Backend

- **Tecnologia:** ASP.NET Core (C#), Entity Framework Core, SQL Server
- **Estrutura:** 3 camadas — Controllers → Servicos → Repositorios → SQL (EF Core)
- **Projetos adicionais:** 3 Windows Services legados (.NET Framework 4.8) — PedidoInternoRecorrente, PrevisaoFluxoCaixa, AjusteDataFaturamento
- **Padrão:** A separação de camadas existe no papel, mas **não é executada corretamente** — os serviços são instanciados manualmente com new dentro dos controllers, eliminando qualquer benefício da injeção de dependência

Frontend

- **Tecnologia:** Vue.js 2 (EOL em dezembro de 2023), Vuex, Vue Router, Axios, Bootstrap-Vue
- **Estrutura:** Pages / Components / Services / Store — estrutura padrão Vue CLI
- **Padrão:** Um único arquivo de serviço (api.service.js, 102 KB) concentra todas as chamadas HTTP do sistema

Banco de Dados

- SQL Server hospedado em Azure VM
- Mapeamento via Entity Framework com IEntityTypeConfiguration por entidade — estrutura adequada
- Scripts SQL encontrados em Documentos/WHEngenharia.sql — presentes, mas não versionados

Separação de Responsabilidades

A separação formal existe, mas é fraca na prática: controllers instanciam serviços diretamente, serviços possuem lógica de apresentação (geração de PDF, Excel), e repositórios contêm regras de negócio. A arquitetura em camadas é mais decorativa do que funcional.

Camada	Componente	Problema
Controller (31 arquivos)	Recebe HTTP, instancia serviços	■ Instancia com new — viola DI
Serviço (29 arquivos)	Regras de negócio	■■ Mistura PDF/Excel/negócio
Repositório (29 arquivos)	Acesso a dados (EF Core)	■■ Contém regras de negócio
SQL (Context + Config)	ORM + schema	■ Adequado

3. Análise do Frontend

Organização do Código

A estrutura de pastas é razoável (pages/components/store/services), mas há problemas relevantes de organização. O arquivo `api.service.js` tem 102 KB e concentra todos os endpoints do sistema — o equivalente a um arquivo de 2.000+ linhas. Qualquer modificação nesse arquivo afeta potencialmente toda a camada de comunicação.

- Nenhuma separação de serviços por domínio (compras, financeiro, RH, etc.)
- Ausência de tipagem — JavaScript puro, sem TypeScript
- 60+ pacotes no `package.json`, incluindo 5 bibliotecas de gráficos simultaneamente
- Vue.js 2 em **EOL (fim de vida)** desde dezembro de 2023 — sem patches de segurança

Estrutura de Serviços (API)

O `api.service.js` usa um padrão de callbacks em vez de Promises modernas:

```
// Padrão atual – callback no estilo Node.js anos 2010
apiService.get("Material", id, (result) => { ... })
```

O Axios já retorna Promises nativas; o wrapper de callback é desnecessário, antiquado e impossibilita composição de chamadas assíncronas e testes unitários.

Performance

■ ■ Aba Materiais/Serviços muito lenta

O método `Get()` sem paginação ainda é utilizado, carregando toda a tabela sem limite.

■ ■ Dupla consulta para busca de texto

`BuscaMaterialPorTexto` executa 2 queries separadas ao banco e faz deduplicação em memória.

■ ■ Sem lazy loading / virtualização

Listas longas são renderizadas integralmente no DOM.

UX e Loading

- Loading inconsistente — algumas telas mostram feedback visual, outras não
- Filtro de compras reportado como não funcional
- Tratamento de erros ad hoc — cada componente trata erros de forma diferente
- Falta de mensagens de erro padronizadas ao usuário

4. Análise do Backend

Injeção de Dependência — Padrão violado em todos os Controllers

Todos os 31 controllers instanciam serviços manualmente, violando o princípio de DI registrado no Startup.cs. Isso impossibilita testes unitários e cria acoplamento rígido:

```
// Em todos os controllers — padrão incorreto:  
_servicoPedidoCompra = new ServicoPedidoCompra(_context, _mapper);  
_servicoObra = new ServicoObra(_context, _mapper);
```

Ausência Total de Logging de Erros

■ Erros em produção são invisíveis

Todos os blocos catch da API descartam a exceção silenciosamente. Não há como diagnosticar falhas sem acesso direto ao servidor.

```
catch (Exception ex)  
{  
    return BadRequest(); // ex não é logado, não é reportado  
}
```

Código Morto / Serviços não Utilizados

- ServicoPedidoCompraFatura — Um único método Post, sem controller dedicado
- WHEngenharia.CustosFixos — Projeto com apenas Program.cs, sem implementação
- ServicesController — Duplica funcionalidade dos 3 Windows Services

Complexidade: ServicoFluxoCaixa.cs

Com 63 KB, é o arquivo mais crítico e complexo do sistema. Concentra a maior parte das regras financeiras, geração de PDF/Excel e lógica de cálculo — tudo misturado em um único arquivo. Contém o bug de duplicação descrito na seção de Problemas Críticos.

Autenticação

O login é implementado como comparação direta de texto em banco de dados — sem hash, sem salt, sem bcrypt:

```
// RepositorioAutenticacao.cs:23  
_context.Usuario.FirstOrDefault(x => x.Login == usuario && x.Senha == senha)
```

Um vazamento do banco de dados expõe as senhas de todos os usuários em texto puro.

5. Banco de Dados

Modelagem

- ~50 entidades mapeadas — cobertura ampla do domínio de negócio
- Uso de `IEntityTypeConfiguration` por entidade — abordagem correta e organizada
- Relacionamentos definidos via Fluent API — positivo
- **LazyLoading desabilitado** — correto, mas exige atenção com includes manuais

Problemas Estruturais Identificados

■ Senhas em texto puro no banco de dados

A coluna `Senha` da tabela `Usuario` armazena senhas sem qualquer hash. Um dump do banco de dados expõe as senhas de todos os usuários diretamente.

■■ Modelo de permissões — 80+ colunas boolean na entidade `Usuario`

As permissões são implementadas como dezenas de colunas boolean diretamente na entidade `Usuario` (ex: `ComprasOrdensCompraValidacaoValidar`, `FinanceiroPedidoInternoRecorrenteCadastrar...`). Isso é inflexível: impossibilita a criação de perfis de acesso, torna inviável reatribuir permissões em massa e gera uma tabela de usuários com mais de 100 colunas.

Versionamento e Scripts

- Um arquivo SQL encontrado em `Documentos/WHEngenharia.sql` — não automatizado
- **Sem migrations do Entity Framework** — mudanças no schema são feitas manualmente
- Sem processo documentado de atualização entre ambientes (dev → produção)

6. Segurança

■ CRÍTICO — Credenciais expostas no repositório

Este é o problema mais grave identificado. Múltiplas credenciais de produção estão commitadas diretamente no repositório e acessíveis a qualquer pessoa com permissão de leitura.

Credenciais Identificadas no Código-Fonte

Arquivo:	appsettings.Development.json appsettings.Test.json
Código:	String de conexão ao SQL Server de produção
Impacto:	Acesso direto ao banco de dados de produção
Arquivo:	ServicoEmail.cs:35
Código:	smtp.Credentials = new NetworkCredential("osvaldo.torezan@solinski.com.br", "#jA8AcZE8EW%UhG")
Impacto:	Comprometimento da conta de e-mail corporativa
Arquivo:	appsettings.json
Código:	"Key": "0D3A!F47aAchC68n4E\$9rB43j2&51%C1Dz@!0u558A24"
Impacto:	Qualquer pessoa pode forjar tokens JWT válidos e autenticar como qualquer usuário

Outros Problemas Críticos de Segurança

■ TTP sem TLS

options.RequireHttpsMetadata = false; e //app.UseHttpsRedirection(); estão desabilitados. config.js do frontend aponta para http:// em produção. Todos os dados (tokens, senhas, dados financeiros) trafegam sem criptografia.

■ senha trafegada como query string (URL)

/api/Usuario/AlterarSenha?id=1&senha;=MINHASENHA — a senha fica exposta em logs de servidor, proxies, histórico de navegador e ferramentas de monitoramento.

■ ndpoints financeiros sem autenticação

[AllowAnonymous] em GeraPedidosInternos, GeraPrevisaoFluxoCaixa e AjustaDataFaturamento — qualquer pessoa na internet pode executar essas operações sem autenticar.

■ ORS completamente aberto

app.UseCors(x => x.AllowAnyMethod().AllowAnyHeader().SetIsOriginAllowed(origin => true)...) sobreescreve a configuração de CorsOrigins do appsettings, permitindo qualquer origem.

■ nfraestrutura em conta pessoal do desenvolvedor

BD hospedado em osvaldotneto-t.eastus2.cloudapp.azure.com — conta pessoal. A empresa não tem controle ou acesso garantido a essa infraestrutura.

7. Performance

■ ■ Carregamento sem paginação

RepositorioMaterial.Get() retorna TODOS os materiais sem limite, paginação ou filtro. Cresce linearmente com o cadastro.

■ ■ Dupla consulta para busca de texto

BuscaMaterialPorTexto executa 2 queries ao banco e deduplicação em memória. Uma única query SQL com ORDER BY seria suficiente.

■ ■ Cálculo de saldo em memória

CalculaSaldoDoMes carrega todos os registros FluxoCaixa do mês em memória e itera dia a dia via LINQ. Para volumes maiores torna-se um gargalo severo.

■ ■ Listas de DEFs hardcoded

DefEhDoFluxoCaixa contém 60+ códigos hardcoded no código C#. Qualquer mudança no plano de contas exige deploy de nova versão.

■ ■ Serialização pesada em log

Windows Service serializa HttpResponseMessage completo: JsonConvert.SerializeObject(response) — gera arquivos de log gigantes.

■ ■ Sem cache em nenhuma camada

Dados de referência (materiais, cidades, cargos) são buscados do banco em cada requisição, sem cache.

8. DevOps e Processo

Pipeline de Deploy

■ Pipeline de Deploy: Inexistente

O deploy é feito manualmente, sem scripts, sem workflows, sem documentação de processo. Não há como reproduzir um deploy de forma confiável por outra pessoa que não o desenvolvedor atual.

Versionamento

- Repositório existe, mas sem estratégia de branches documentada
- Ausência de **.gitignore adequado** no backend — credenciais foram commitadas
- O repositório está em REPO-TEMPORARIO — indicativo de falta de governança
- Sem tags de release ou controle de versão semântico

Controle de Acesso

- Hospedagem em conta do desenvolvedor, não da empresa
- Sem separação entre ambientes (dev / staging / produção) no processo de deploy
- Acesso ao banco de dados de produção disponível para quem lê o repositório

Qualidade do Processo

Item	Status
Testes automatizados (unitários)	■ Ausente
Testes de integração	■ Ausente
Testes E2E	■ Ausente
Pipeline CI (build automático)	■ Ausente
Pipeline CD (deploy automatizado)	■ Ausente
Análise estática de segurança (SAST)	■ Ausente
Logging centralizado	■ Ausente
Documentação de API (Swagger)	■ Presente (Swagger básico)
Documentação de deploy	■ Ausente
Migrations de banco de dados	■ Ausente

9. Problemas Críticos (ALTA PRIORIDADE)

Lista de problemas que precisam ser corrigidos com urgência, ordenados por impacto:

#	Problema	Impacto	Arquivo/Local
1	Credenciais de BD no repositório	Acesso direto ao banco de produção	appsettings.Development.json appsettings.Test.json
2	Senha SMTP hardcoded	Comprometimento de conta de e-mail	ServicoEmail.cs:35
3	JWT signing key no repositório	Forge de tokens — acesso total ao sistema	appsettings.json
4	Senhas em texto puro (BD)	Vazamento expõe senhas de todos usuários	RepositorioAutenticacao.cs:23
5	Senha como query string (URL)	Senha visível em logs e proxies	UsuarioController.cs:132
6	Endpoints financeiros sem auth	Qualquer pessoa executa operações financeiras	ServicesController.cs:36, 52, 68
7	Bug: cálculo duplicado FluxoCaixa	DEF 03.33 somado 3x — relatórios incorretos	ServicoFluxoCaixa.cs:85, 88, 90
8	HTTP sem TLS (produção)	Dados financeiros e senhas sem criptografia	Program.cs + Startup.cs + config.js
9	Infra em conta pessoal	Risco de descontinuidade do negócio	Azure – osvaldotneto-t...
10	Zero logging de erros	Falhas em produção são invisíveis	Todos os controllers
11	Duplicação Jobs vs. API	Risco de double-execution financeiro	ServicesController + Services.*

10. Melhorias Recomendadas

■ Imediato — Segurança (executar antes de qualquer outra coisa)

- 1. Rotacionar TODAS as credenciais expostas agora — BD, SMTP, JWT key. Varrer histórico do repositório com git-filter-repo para remoção permanente.
- 2. Transferir a infraestrutura Azure para conta corporativa da empresa.
- 3. Remover credenciais do código — usar Azure Key Vault ou variáveis de ambiente. Nunca versionar appsettings.Development.json com credenciais reais.
- 4. Implementar hash de senhas — substituir comparação direta por bcrypt ou PBKDF2.
- 5. Habilitar HTTPS obrigatório — remover comentário do UseHttpsRedirection() e configurar certificado SSL.
- 6. Proteger endpoints do ServicesController com API Key ou Secret dedicado.
- 7. Corrigir bug do FluxoCaixa — remover dupla inclusão do DEF 03.33 nas linhas 85, 88 e 90 do ServicoFluxoCaixa.cs.

■ Curto Prazo (3-6 meses)

- 1. Implementar pipeline CI/CD (GitHub Actions) com build, lint e deploy controlado.
- 2. Definir estratégia de branches: feature/* → develop (via PR) → main (produção protegida).
- 3. Adicionar logging centralizado — Serilog ou Application Insights.
- 4. Refatorar injeção de dependência — registrar serviços no Startup.cs e injetar via construtor em todos os controllers.
- 5. Corrigir fluxo de troca de senha — enviar via body HTTPS, nunca query string.
- 6. Adicionar testes unitários — priorizar ServicoFluxoCaixa e ServicoSolicitacaoCompra.
- 7. Validar data de entrega no passado em ServicoSolicitacaoCompra.Post().

■ Médio Prazo (6-12 meses)

- 1. Migrar para Vue 3 — Vue 2 está em EOL. Planejar migração incremental.
- 2. Reduzir dependências do frontend — auditar e remover bibliotecas não utilizadas.
- 3. Implementar RBAC adequado — substituir as 80+ colunas boolean por modelo de roles.
- 4. Eliminar ambiguidade entre Services e Jobs — decidir padrão único de orquestração.
- 5. Implementar Entity Framework Migrations para rastrear mudanças de schema.
- 6. Fragmentar api.service.js por domínio (compras, financeiro, RH...).
- 7. Adicionar paginação em todos os endpoints que retornam listas sem limite.

11. Classificação Final do Sistema



ALTO RISCO

Justificativa

O sistema possui funcionalidade de negócio implementada e operante, cobrindo um domínio complexo (ERP de engenharia com compras, financeiro, obras, RH). A arquitetura base existe e há sinais de capacidade técnica razoável. Entretanto, os problemas de segurança são de natureza **crítica e ativa** — não são riscos futuros, são vulnerabilidades presentes hoje:

■ Credenciais de produção no repositório

Qualquer pessoa com acesso ao repositório pode conectar ao banco de dados de produção diretamente. Esta é a vulnerabilidade mais grave possível em qualquer sistema.

■ JWT signing key exposta

Permite que qualquer pessoa autentique no sistema como qualquer usuário, sem precisar de senha.

■ Senhas em texto puro

Elimina qualquer proteção residual em caso de vazamento do banco de dados.

■ Bug no Fluxo de Caixa

Compromete a confiabilidade de relatórios financeiros que embasam decisões de gestão.

■ Dependência total do desenvolvedor atual

Infraestrutura em conta pessoal, sem documentação de deploy, sem processo de transferência — pode paralisar a operação em caso de rescisão contratual.

O Sistema Seria ■ Médio Risco se:

- As credenciais fossem removidas e rotacionadas imediatamente
- A infraestrutura fosse transferida para conta corporativa da empresa
- O bug do FluxoCaixa fosse corrigido
- HTTPS fosse habilitado obrigatoriamente

Relatório gerado com base na análise direta do código-fonte disponível no repositório. Itens que não puderam ser verificados por ausência de acesso ao ambiente de execução estão sinalizados explicitamente no texto. Abril de 2026.